Agenda de Contactos Persistente

Leitura e Escrita de Ficheiros

Bernardo Toninho
IST and INESC-ID

Fundamentos de Programação
2026–2027
Baseado em slides de Luís Caires

Outline

1. O Projecto
2. Representar a Agenda num Ficheiro
3. Ler a Agenda
4. Guardar a Agenda
5. Compor o Programa
6. Aplicação Completa
7. Síntese

A Agenda Entre Duas Execuções

Na Aula 8 construímos uma agenda em memória. Enquanto o programa está a correr, podemos acrescentar contactos e consultar telefones.

O problema

Quando o programa termina, a agenda desaparece. Na execução seguinte voltamos a começar com um dicionário vazio.

A Agenda Entre Duas Execuções

Na Aula 8 construímos uma agenda em memória. Enquanto o programa está a correr, podemos acrescentar contactos e consultar telefones.

O problema

Quando o programa termina, a agenda desaparece. Na execução seguinte voltamos a começar com um dicionário vazio.

A nova funcionalidade

Guardar a agenda num ficheiro e recuperá-la mais tarde.

Funcionalidades do Programa

O programa completo deve permitir:

- acrescentar contactos e telefones;
- consultar, listar e remover dados;
- guardar o estado actual num ficheiro;
- carregar uma agenda anteriormente guardada;
- comunicar ficheiros inexistentes e dados inválidos.

Foco desta aula

As operações sobre a agenda já existem. Vamos construir a fronteira entre os valores Python e a sua representação num ficheiro de texto.

Ponto de Partida: Dados em Memória

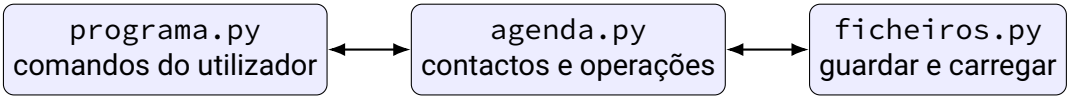
```python
from dataclasses import dataclass

@dataclass
class Contacto:
    nome: str
    telefones: dict[str, str]

Agenda = dict[str, Contacto]
```

- A agenda localiza cada contacto pelo nome.
- Cada contacto associa etiquetas a números de telefone.
- As funções da Aula 8 preservam os invariantes desta representação.

Organização do Projecto



- `agenda.py` não precisa de conhecer ficheiros nem comandos.
- `ficheiros.py` reutiliza as operações públicas da agenda.
- `programa.py` coordena a interacção e comunica erros.

Outline

1. O Projecto
2. Representar a Agenda num Ficheiro
3. Ler a Agenda
4. Guardar a Agenda
5. Compor o Programa
6. Aplicação Completa
7. Síntese

O Formato Faz Parte do Programa

Um ficheiro de texto contém caracteres, não contém directamente dicionários ou valores `Contacto`.

Decisões necessárias

- Como representamos o início de um contacto?
- Como associamos uma etiqueta e um número ao contacto certo?
- Como reconhecemos uma linha inválida?

Guardar e carregar só são operações inversas quando seguem exactamente o mesmo formato.

Um Formato com Dois Tipos de Registo

```
CONTACTO;Ana
TELEFONE;Ana;casa;213456789
TELEFONE;Ana;movel;912345678
CONTACTO;Bruno
TELEFONE;Bruno;movel;934567890
```

Um Formato com Dois Tipos de Registo

```
CONTACTO;Ana
TELEFONE;Ana;casa;213456789
TELEFONE;Ana;movel;912345678
CONTACTO;Bruno
TELEFONE;Bruno;movel;934567890
```

- `CONTACTO;nome` cria um contacto, mesmo que não tenha telefones.
- `TELEFONE;nome;etiqueta;numero` acrescenta um telefone.
- O ponto e vírgula separa os campos de cada registo.

Contrato do Formato

Um ficheiro válido respeita as seguintes regras:

- cada linha contém um registo completo;
- um registo `CONTACTO` tem exactamente dois campos;
- um registo `TELEFONE` tem exactamente quatro campos;
- cada telefone aparece depois do respectivo contacto;
- nomes, etiquetas e números não são vazios nem contêm ponto e vírgula.

Consequência

O programa pode validar uma linha antes de alterar a agenda.

Outline

1. O Projecto
2. Representar a Agenda num Ficheiro
3. Ler a Agenda
4. Guardar a Agenda
5. Compor o Programa
6. Aplicação Completa
7. Síntese

Abrir o Ficheiro

```
1 with open("contactos.txt", "r", encoding="utf-8") as ficheiro:
2     texto = ficheiro.read()
3
4 print(texto)
```

Abrir o Ficheiro

```
1 with open("contactos.txt", "r", encoding="utf-8") as ficheiro:
2     texto = ficheiro.read()
3
4 print(texto)
```

- `"r"` abre o ficheiro para leitura.
- `encoding="utf-8"` define como os caracteres são interpretados.
- Ao sair do bloco `with`, o ficheiro é fechado automaticamente.

Percorrer Linha a Linha

```
1 with open("contactos.txt", "r", encoding="utf-8") as ficheiro:
2     for linha in ficheiro:
3         print(repr(linha))
```

Percorrer Linha a Linha

```
1 with open("contactos.txt", "r", encoding="utf-8") as ficheiro:
2     for linha in ficheiro:
3         print(repr(linha))
```

- O objecto ficheiro produz uma linha de cada vez.
- Não precisamos de guardar todo o texto numa única string.
- A linha inclui normalmente o carácter final `"\n"`.

A Mudança de Linha Também é um Carácter

Se a primeira linha do ficheiro for:

```
CONTACTO;Ana
```

o valor recebido pelo programa é:

```
1 linha == "CONTACTO;Ana\n"
```

A Mudança de Linha Também é um Carácter

Se a primeira linha do ficheiro for:

```
CONTACTO;Ana
```

o valor recebido pelo programa é:

```
1 linha == "CONTACTO;Ana\n"
```

Antes de separar os campos, retiramos apenas a mudança de linha:

```
1 limpa = linha.rstrip("\n")
```

Exercício: Valores Produzidos pelo Ficheiro

O ficheiro contém exactamente:

```
CONTACTO;Ana
TELEFONE;Ana;casa;213456789
```

Considere:

```
1 with open("contactos.txt", "r", encoding="utf-8") as ficheiro:
2     for linha in ficheiro:
3         print(repr(linha.rstrip("\n")))
```

Que duas strings são apresentadas no ecrã?

Resultado Esperado: Linhas sem \n

Problema: determinar os valores produzidos depois de remover a mudança de linha final.

Resultado Esperado: Linhas sem \n

Problema: determinar os valores produzidos depois de remover a mudança de linha final.

```
'CONTACTO;Ana'
'TELEFONE;Ana;casa;213456789'
```

`rstrip("n")` não altera os separadores entre os campos.

Separar os Campos

```
1 linha = "TELEFONE;Ana;casa;213456789\n"
2 limpa = linha.rstrip("\n")
3 campos = limpa.split(";")
4
5 print(campos)
6 # ['TELEFONE', 'Ana', 'casa', '213456789']
```

Separar os Campos

```
1 linha = "TELEFONE;Ana;casa;213456789\n"
2 limpa = linha.rstrip("\n")
3 campos = limpa.split(";")
4
5 print(campos)
6 # ['TELEFONE', 'Ana', 'casa', '213456789']
```

Ainda não alterámos a agenda

Primeiro interpretamos e validamos os campos. Só depois chamamos uma operação da agenda.

Sinalizar uma Linha Inválida

Quando uma linha não respeita o contrato, a função não consegue produzir uma actualização válida.

- `raise ValueError("explicacao")` interrompe a operação.
- A mensagem descreve a regra que falhou.
- A função que lê o ficheiro pode acrescentar o número da linha.

Antes de alterar

Todas as verificações devem acontecer antes de modificar a agenda.

Interpretar um Registo de Contacto

```
1 def aplica_contacto(agenda: Agenda,
2                      campos: list[str]) -> None:
3     if len(campos) != 2 or campos[1] == "":
4         raise ValueError("registo CONTACTO invalido")
5
6     nome = campos[1]
7     if nome in agenda:
8         raise ValueError("contacto repetido")
9     adiciona_contacto(agenda, nome)
```

O registo só altera a agenda depois de todas as verificações.

Interpretar um Registo de Telefone

```
1  def aplica_telefone(agenda: Agenda,
2                      campos: list[str]) -> None:
3      if len(campos) != 4:
4          raise ValueError("registo TELEFONE invalido")
5      _, nome, etiqueta, numero = campos
6      if nome not in agenda:
7          raise ValueError("contacto ainda nao definido")
8      if etiqueta == "" or numero == "":
9          raise ValueError("etiqueta ou numero vazio")
10     adiciona_telefone(agenda, nome, etiqueta, numero)
```

Escolher a Operação pelo Primeiro Campo

```python
def aplica_linha(agenda: Agenda, linha: str) -> None:
    campos = linha.rstrip("\n").split(";")
    tipo = campos[0]

    if tipo == "CONTACTO":
        aplica_contacto(agenda, campos)
    elif tipo == "TELEFONE":
        aplica_telefone(agenda, campos)
    else:
        raise ValueError("tipo de registo desconhecido")
```

Cada ramo delega a validação detalhada na função correspondente.

Construir a Agenda Gradualmente

```python
def carrega_agenda(nome_ficheiro: str) -> Agenda:
    agenda: Agenda = {}
    with open(nome_ficheiro, "r", encoding="utf-8") as ficheiro:
        for linha in ficheiro:
            aplica_linha(agenda, linha)
    return agenda
```

- Cada linha válida acrescenta informação à mesma agenda.
- A ordem do ficheiro garante que o contacto surge antes dos telefones.
- O resultado só é devolvido depois de processar todo o ficheiro.

Acrescentar o Número da Linha ao Erro

```
1  def carrega_agenda(nome_ficheiro: str) -> Agenda:
2      agenda: Agenda = {}
3      with open(nome_ficheiro, "r", encoding="utf-8") as ficheiro:
4          for numero, linha in enumerate(ficheiro, start=1):
5              try:
6                  aplica_linha(agenda, linha)
7              except ValueError as erro:
8                  raise ValueError(
9                      f"linha {numero}: {erro}") from erro
10     return agenda
```

- `enumerate(ficheiro, start=1)` produz pares com o número e a linha.
- O contexto identifica onde falhou o formato sem esconder a causa.

Um Ficheiro Pode Não Existir

```
1 try:
2     agenda = carrega_agenda("contactos.txt")
3 except FileNotFoundError:
4     print("O ficheiro nao existe.")
5 except ValueError as erro:
6     print("Agenda invalida:", erro)
```

Um Ficheiro Pode Não Existir

```
1 try:
2     agenda = carrega_agenda("contactos.txt")
3 except FileNotFoundError:
4     print("O ficheiro nao existe.")
5 except ValueError as erro:
6     print("Agenda invalida:", erro)
```

Excepções diferentes

`FileNotFoundError` descreve o acesso ao ficheiro. `ValueError` descreve conteúdo que não respeita o contrato.

O Papel do `with`

Se `aplica_linha` levantar uma excepção dentro do bloco:

- o percurso do ficheiro termina;
- o ficheiro é fechado automaticamente;
- a excepção continua até uma função que a consiga tratar.

Vantagem

Não precisamos de repetir `close()` em todos os caminhos de execução.

Outline

1. O Projecto
2. Representar a Agenda num Ficheiro
3. Ler a Agenda
4. Guardar a Agenda
5. Compor o Programa
6. Aplicação Completa
7. Síntese

Abrir para Escrita

```
1 with open("contactos.txt", "w", encoding="utf-8") as ficheiro:
2     ficheiro.write("CONTACTO;Ana\n")
```

Abrir para Escrita

```
1 with open("contactos.txt", "w", encoding="utf-8") as ficheiro:
2     ficheiro.write("CONTACTO;Ana\n")
```

- `"w"` cria o ficheiro ou substitui o conteúdo anterior.
- `write` escreve exactamente os caracteres recebidos.
- A mudança de linha tem de aparecer explicitamente no texto.

Escrever um Contacto Completo

```
1 def escreve_contacto(ficheiro, contacto: Contacto) -> None:
2     ficheiro.write(f"CONTACTO;{contacto.nome}\n")
3     for etiqueta, numero in contacto.telefones.items():
4         ficheiro.write(
5             f"TELEFONE;{contacto.nome};{etiqueta};{numero}\n"
6         )
```

- A primeira linha permite recuperar contactos sem telefones.
- As linhas seguintes recuperam as associações do dicionário interior.

Guardar Todos os Contactos

```
1 def guarda_agenda(nome_ficheiro: str,
2                   agenda: Agenda) -> None:
3     with open(nome_ficheiro, "w", encoding="utf-8") as ficheiro:
4         for contacto in agenda.values():
5             escreve_contacto(ficheiro, contacto)
```

Guardar Todos os Contactos

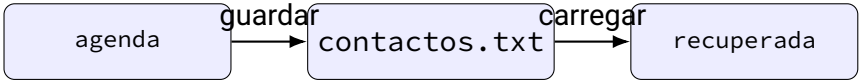
```
1 def guarda_agenda(nome_ficheiro: str,
2                   agenda: Agenda) -> None:
3     with open(nome_ficheiro, "w", encoding="utf-8") as ficheiro:
4         for contacto in agenda.values():
5             escreve_contacto(ficheiro, contacto)
```

Uma fronteira pequena

Só esta função abre o ficheiro. A função auxiliar escreve um contacto no formato definido pelo projecto.

Guardar e Voltar a Carregar

O teste central da persistência é uma viagem de ida e volta:



Propriedade esperada

```
recuperada == agenda
```

Exercício: Conteúdo Produzido

Considere:

```
1 agenda = {
2     "Ana": Contacto("Ana", {"casa": "213456789"}),
3     "Bruno": Contacto("Bruno", {})
4 }
5 guarda_agenda("contactos.txt", agenda)
```

Que linhas ficam guardadas em `contactos.txt`? O contacto sem telefones continua representado?

Resultado Esperado: Conteúdo Guardado

Problema: aplicar o formato a um contacto com telefone e a um contacto sem telefones.

Resultado Esperado: Conteúdo Guardado

Problema: aplicar o formato a um contacto com telefone e a um contacto sem telefones.

```
CONTACTO;Ana
TELEFONE;Ana;casa;213456789
CONTACTO;Bruno
```

Sim. A linha `CONTACTO;Bruno` preserva o contacto vazio.

Outline

1. O Projecto
2. Representar a Agenda num Ficheiro
3. Ler a Agenda
4. Guardar a Agenda
5. Compor o Programa
6. Aplicação Completa
7. Síntese

Um Percurso Completo

```python
def actualiza_agenda(origem: str, destino: str) -> None:
    agenda = carrega_agenda(origem)
    adiciona_contacto(agenda, "Carla")
    adiciona_telefone(
        agenda, "Carla", "movel", "965432100")
    guarda_agenda(destino, agenda)
```

1. ler caracteres do ficheiro de origem;
2. reconstruir e alterar valores Python;
3. escrever o novo estado no ficheiro de destino.

Tratar Erros na Fronteira do Programa

```python
def main() -> None:
    try:
        actualiza_agenda("contactos.txt", "contactos-novos.txt")
    except FileNotFoundError:
        print("Nao foi possivel abrir a agenda.")
    except ValueError as erro:
        print("O ficheiro tem dados invalidos:", erro)
```

- As funções interiores detectam e descrevem os problemas.
- `main` decide a mensagem que o utilizador recebe.

Responsabilidades dos Módulos

`agenda.py` define `Contacto`, `Agenda` e as operações sobre os dados em memória.

`ficheiros.py` define o formato e implementa `carrega_agenda` e `guarda_agenda`.

`programa.py` recebe pedidos, compõe operações e apresenta mensagens ao utilizador.

Dependências

O módulo de domínio continua utilizável sem ficheiros e sem interacção.

Testar as Partes do Projecto

```
1  agenda: Agenda = {}
2  aplica_linha(agenda, "CONTACTO;Ana\n")
3  aplica_linha(agenda, "TELEFONE;Ana;casa;213456789\n")
4
5  assert agenda["Ana"].telefones["casa"] == "213456789"
6
7  try:
8      aplica_linha(agenda, "TELEFONE;Rui;casa;210000000\n")
9      assert False
10 except ValueError:
11     pass
```

Podemos testar a interpretação de linhas sem depender de interacção.

Outline

1. O Projecto
2. Representar a Agenda num Ficheiro
3. Ler a Agenda
4. Guardar a Agenda
5. Compor o Programa
6. Aplicação Completa
7. Síntese

Interface de Comandos

O programa interactivo acrescenta uma camada sobre as funções existentes:

- `ADD nome` cria um contacto;
- `PHONE nome etiqueta numero` define um telefone;
- `GET nome etiqueta` consulta um telefone;
- `REMOVE nome etiqueta` remove um telefone;
- `LIST` apresenta todos os contactos;
- `SAVE ficheiro` e `LOAD ficheiro` gerem a persistência;
- `QUIT` termina a execução.

Para simplificar o formato dos comandos, nomes e argumentos não contêm espaços.

De uma Linha a um Comando

```
1 linha = input("> ")
2 partes = linha.split()
3
4 if partes == []:
5     print("Comando vazio.")
6 else:
7     comando = partes[0]
8     argumentos = partes[1:]
```

De uma Linha a um Comando

```
1 linha = input("> ")
2 partes = linha.split()
3
4 if partes == []:
5     print("Comando vazio.")
6 else:
7     comando = partes[0]
8     argumentos = partes[1:]
```

`split()` sem argumento separa em espaços consecutivos e não produz campos vazios nas extremidades.

Comandos que Alteram a Agenda

```
1  def tenta_alterar(agenda: Agenda, comando: str,
2                    args: list[str]) -> bool:
3      if comando == "ADD" and len(args) == 1:
4          adiciona_contacto(agenda, args[0])
5          return True
6      if comando == "PHONE" and len(args) == 3:
7          nome, etiqueta, numero = args
8          if nome not in agenda:
9              raise ValueError("contacto inexistente")
10         adiciona_telefone(agenda, nome, etiqueta, numero)
11         return True
12     return False
```

Comandos de Consulta e Remoção

```
1  def executa_agenda(agenda: Agenda, comando: str,
2                     args: list[str]) -> None:
3      if tenta_alterar(agenda, comando, args):
4          return
5      if comando == "GET" and len(args) == 2:
6          print(procura_telefone(agenda, args[0], args[1]))
7      elif comando == "REMOVE" and len(args) == 2:
8          if not remove_telefone(agenda, args[0], args[1]):
9              raise ValueError("telefone inexistente")
10     elif comando == "LIST" and args == []:
11         lista(agenda)
12     else:
13         raise ValueError("comando ou argumentos invalidos")
```

Integrar SAVE e LOAD

```
1 def executa(agenda: Agenda, comando: str,
2             args: list[str]) -> Agenda:
3     if comando == "SAVE" and len(args) == 1:
4         guarda_agenda(args[0], agenda)
5     elif comando == "LOAD" and len(args) == 1:
6         agenda = carrega_agenda(args[0])
7     else:
8         executa_agenda(agenda, comando, args)
9     return agenda
```

LOAD devolve outra agenda; por isso o chamador deve guardar o resultado.

Listar os Contactos

```
1 def lista(agenda: Agenda) -> None:
2     for nome in agenda:
3         contacto = agenda[nome]
4         print(nome)
5         for etiqueta, numero in contacto.telefones.items():
6             print("  ", etiqueta, numero)
```

Listar os Contactos

```python
def lista(agenda: Agenda) -> None:
    for nome in agenda:
        contacto = agenda[nome]
        print(nome)
        for etiqueta, numero in contacto.telefones.items():
            print("  ", etiqueta, numero)
```

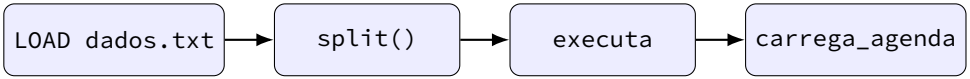
Esta função apresenta a estrutura em memória. Não reutiliza o formato do ficheiro, porque o destinatário e o objectivo são diferentes.

O Ciclo de Comandos

```python
def ciclo_comandos() -> None:
    agenda: Agenda = {}
    while True:
        partes = input("> ").split()
        if partes == []:
            continue
        if partes == ["QUIT"]:
            return
        try:
            agenda = executa(agenda, partes[0], partes[1:])
        except (ValueError, FileNotFoundError) as erro:
            print("Erro:", erro)
```

O ciclo continua depois de um pedido inválido ou de uma leitura falhada.

Percurso de um Comando LOAD



- O texto do utilizador determina a operação pedida.
- O ficheiro determina os valores da nova agenda.
- Uma excepção regressa ao ciclo, que apresenta a mensagem.

Exercício: Um Comando Desconhecido

Considere a agenda vazia e a linha:

```
EMAIL Ana ana@example.org
```

Siga o percurso desta entrada por `split`, `executa` e `ciclo_comandos`. O programa termina? A agenda é alterada? O que aparece no ecrã?

Resultado Esperado: Comando Desconhecido

Problema: determinar o efeito de `EMAIL Ana ana@example.org`, que não está implementado.

Resultado Esperado: Comando Desconhecido

Problema: determinar o efeito de `EMAIL Ana ana@example.org`, que não está implementado.

- `split()` produz `["EMAIL", "Ana", "ana@example.org"]`.
- `executa` levanta `ValueError`.
- O **`except`** apresenta a mensagem e preserva a agenda vazia.
- O ciclo pede o comando seguinte.

Linhas Vazias e Comentários

Podemos estender o formato para aceitar ficheiros mais fáceis de editar:

```
for numero, linha in enumerate(ficheiro, start=1):
    limpa = linha.strip()
    if limpa == "" or limpa.startswith("#"):
        continue
    try:
        aplica_linha(agenda, limpa)
    except ValueError as erro:
        raise ValueError(f"linha {numero}: {erro}") from erro
```

Esta extensão altera o contrato e exige testes adicionais.

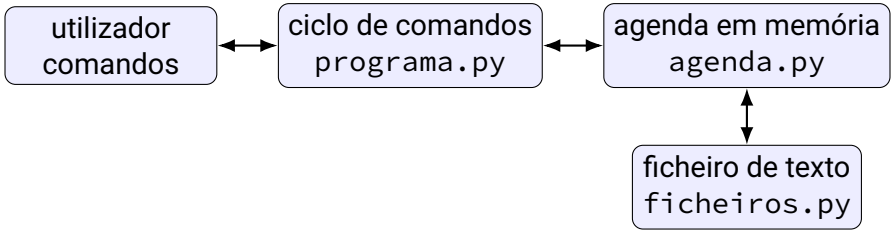
Caminhos com `pathlib`

```python
from pathlib import Path

caminho = Path("dados") / "contactos.txt"
with caminho.open("r", encoding="utf-8") as ficheiro:
    for linha in ficheiro:
        print(linha.rstrip("\n"))
```

`Path` permite construir caminhos sem concatenar manualmente os separadores das pastas.

O Programa Completo



Cada fronteira tem um contrato: sintaxe dos comandos, invariantes da agenda e formato do ficheiro.

Outline

1. O Projecto
2. Representar a Agenda num Ficheiro
3. Ler a Agenda
4. Guardar a Agenda
5. Compor o Programa
6. Aplicação Completa
7. Síntese

Conclusão

Hoje vimos

- leitura e escrita de ficheiros de texto com `with`
- reconstrução e serialização de uma agenda de contactos
- separação entre domínio, persistência e interacção

A seguir

- strings como sequências de caracteres
- operações sistemáticas de processamento textual
- normalização, pesquisa e contagem em texto